Design Principles

SOLID

SOLID

- SOLID is one of the most popular sets of design principles in object-oriented software development.
- Adopting these practices can also contribute to avoiding code smells, refactoring code, and Agile or Adaptive software development.

SOLID(Cont.)

- Design principles encourage us to create more maintainable, understandable, and flexible software.
- As our applications grow in size, we can reduce their complexity by using design principles.

SOLID Principles

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Single Responsibility

- A class should only have one responsibility.
- It should only have one reason to change.

```java
//Before Refactoring
public class User {
    public void login(String username, String password) {
        // Authenticate user
    }

    public void sendEmail(String to, String subject, String body) {
        // Send email
    }
}
```

Single Responsibility

```java
// After refactoring
public class User {
    public void login(String username, String password) {
        // Authenticate user
    }
}

class EmailService {
    public void sendEmail(String to, String subject, String body) {
        // Send email
    }
}
```

Open/Closed Principle

- Open for extension, closed for modification.
- The OCP states that software entities (classes, modules, functions, etc.) should be open for extension but closed for modification.
- In other words, you should be able to add new functionality to your code without changing the existing code.

```
class Shape {
    public double calculateArea(String shapeType, double param1, double param2) {
        double area = 0.0;
        if (shapeType.equalsIgnoreCase( anotherString: "circle")) {
            double radius = param1;
            area = 3.14 * radius * radius;
        } else if (shapeType.equalsIgnoreCase( anotherString: "rectangle")) {
            double width = param1;
            double height = param2;
            area = width * height;
        }

        return area;
    }
}
```

Before Refactoring

```java
abstract class Shape {
    public abstract double calculateArea();
}
class Circle extends Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return 3.14 * radius * radius;
    }
}
```

```java
class Rectangle extends Shape {
    private double width;
    private double height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }
}
```

After Refactoring

Liskov Substitution Principle

- The Liskov Substitution Principle states that subclasses should be substitutable for their base classes.
- This means that, given that class B is a subclass of class A, We should be able to pass an object of class B to any method that expects an object of class A and the method should not give any weird output in that case.
- Child class in replacement of Parent Class.

```
class Bird{
    public void fly(){}
}
class Duck extends Bird{}
class Ostrich extends Bird{}
```

Before Refactoring

```
class Bird{}
class FlyingBirds extends Bird{
    public void fly(){}
}
class Duck extends FlyingBirds{}
class Ostrich extends Bird{}
```

After Refactoring

Interface Segregation Principle

- The principle states that many client-specific interfaces are better than one general-purpose interface.
- Clients should not be forced to implement a function they do not need.

```java
interface Vehicle {
    void setPrice(double price);
    void start();
    void stop();
    void fly();
}
```

Breaks Interface Segregation Principle

```java
class Car implements Vehicle {
    double price;
    @Override
    public void setPrice(double price) {
        this.price = price;
    }

    @Override
    public void start(){}
    @Override
    public void stop(){}
    @Override
    public void fly(){}
}
```

```java
interface Vehicle {
    void setPrice(double price);
}

interface Movable {
    void start();
    void stop();
}

interface Flyable {
    void fly();
}
```

Interface Segregation

```java
class Car implements Vehicle, Movable {
    double price;

    @Override
    public void setPrice(double price) {

        this.price = price;
    }

    @Override
    public void start(){
        // Implementation
    }
    @Override
    public void stop(){
        // Implementation
    }
}
```

```java
class Aeroplane implements Vehicle, Movable, Flyable {
    double price;

    @Override
    public void setPrice(double price) {

        this.price = price;
    }

    @Override
    public void start(){
        // Implementation
    }
    @Override
    public void stop(){
        // Implementation
    }
    @Override
    public void fly(){
        // Implementation
    }
}
```

```java
public class VehicleBuilder {
    public static Car buildCar(){
        Car car = new Car();
        car.setPrice(15.00);
        car.start();
        return car;
    }
    public static Aeroplane buildAeroPlane(){
        Aeroplane aeroplane = new Aeroplane();
        aeroplane.setPrice(25.00);
        aeroplane.start();
        aeroplane.fly();
        return aeroplane;
    }
}
```

Interface Segregation

Dependency Inversion principle

- The Dependency Inversion Principle (DIP) states that high-level modules should not depend on low-level modules.
- Instead, both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.
- In other words, your code should depend on interfaces or abstract classes, rather than concrete implementations.

```
class Notification {
    private MsgNotification msgNotification;
    private CallNotification callNotification;

    public Notification() {
        this.msgNotification = new msgNotification();
        this.callNotification = new callNotification();
    }

    void notify() {
        msgNotification.notify();
        callNotification.notify();
    }
}

class MsgNotification {
    void notify() {

    }
}

class CallNotification {
    void notify() {

    }
}
```

```java
interface INotification {
    void notify();
}

class Notification {
    private INotification notification;

    public Notification(INotification notification) {
        this.notification = notification;
    }

    public void notify() {
        notification.notify();
    }
}

class MsgNotification implements INotification {
    @Override
    public void notify() { }
}

class CallNotification implements INotification {
    @Override
    public void notify() { }
}
```